AMU | Language, Mind, & Tech
Programming in Python
Ali Asgari

Bloodhound\n

Regular Expression

a specialized *mini-language* used to describe search patterns in text

it allows you to perform complex text operations with a single line of code.
It is primarily used for:

Validation:

Ensuring a string follows a specific format

(e.g., "Is this a valid email address or phone number?")

Extraction:

Pulling specific pieces of information out of a large document

(e.g., "Find all the dates mentioned in these 1,000 transcripts").

Transformation:

Mass-cleaning data

(e.g., "Replace all inconsistent date formats with a standardized YYYY-MM-DD format").

The re Search Functions

Method	Syntax	Technical Description
re.findall()	re.findall(pattern, text)	Scans the whole text for **all** matches. Returns a **list of strings**.
re.search()	re.search(pattern, text)	Scans for the **first** match. Returns a "Match Object" (a report) or None.
.group()	match_object.group()	A detailed "report" created by re.search. It contains information about the find. .group() is a command used to pull the actual text out of a Match Object.
\d	r"\d"	**The Scent:** \d+ means one or more numbers. (as a pattern to look for)

The re Search Functions

```
import re

# Data: A log from a cognitive psychology experiment
log_data = "Subject_A: 450ms, Subject_B: 320ms, Subject_C: 510ms"

# 1. Detection: Find the FIRST reaction time
# The dog finds the first "scent" and creates a report (Match Object)
first_report = re.search(r"\d+ms", log_data)

if first_report:
    # We summon the text from the report using .group()
    print(f"First reaction time found: {first_report.group()}")

# 2. Trailing: Find ALL reaction times
# The dog tracks every footprint and puts them in a list
all_times_list = re.findall(r"\d+ms", log_data)

print(f"All reaction times found: {all_times_list}")
```

The re Search Functions

Side Quest!

You have a file with 1 million lines. You only need to know if the word "ERROR" appears once. Which function should you use?

Your reward : 1pt

The Scent and the Perimeter

Component	Syntax	Technical Description
Literal	**abc**	Matches the **exact** sequence of characters. Case-sensitive.
Wildcard	**.**	The "Anything" Scent. Matches **any single character** (letter, digit, space, symbol).
Escape	****	The "Power-Off" switch. Turns magic symbols into normal text (e.g., \. is a real dot).
Word Char	**\w**	Matches any **Alphanumeric** character (a-z, A-Z, 0-9) and **underscores** (_).
Digit	**\d**	Matches any single **number** from 0 to 9.
Whitespace	**\s**	Matches **spaces, tabs, and newlines**. The "invisible" scent.
Newline	**\n**	Matches the specific character that starts a **new line**.
Start Anchor	**^**	Forces the match to be at the very **beginning** of the string.
End Anchor	**$**	Forces the match to be at the very **end** of the string.

The Scent and the Perimeter

```
import re
comment = "I love this part of data!"

# Literal: Looking for the exact word 'data'
print(len(re.findall(r"data", comment)))

# Wildcard (.): Looking for 'p', then 2 ANY character, then 't'
# This will catch 'p..t' (the literal dot counts as 'any character')
print(re.findall(r"p..t", comment))
```

```
comment = "Check the site neuro.com"

# Searching for a literal dot between before 'com'
# Without the \, the dot would be a wildcard.
match = re.search(r"\w+\.com", comment)
print(match.group())
```

```
tweet = "User @Dr_Brain7 detected 10 errors.“

# \w+ finds a string of username
print(re.findall(r"@\w+", tweet))

# \d+ finds the numbers
print(re.findall(r"\b\d+\b", tweet))

# \s finds the spaces
print(len(re.findall(r"\s", tweet)))
```

```
post = "RT: This study is amazing!"

if re.search(r"^RT", post):
    print("This is a Retweet.")

if re.search(r"!$", post):
    print("The user is very excited!")
```

The Scent and the Perimeter

Side Quest!

I want to find a serial code that is exactly five numbers long and nothing else (e.g., "12345"). How do I use anchors to make sure the dog doesn't bark at "1234567"?

Your reward : 4pt

```
r"\b\d\d\d\d\d\b"
```

Scent Discrimination & Stamina

Component	Syntax	Technical Description
Character Set	**[aeiou]**	Matches **any one** character inside the brackets (e.g., any vowel).
Range	**[a-z]**	Matches any character in the specified alphabet or number range.
Negated Set	**[^abc]**	Matches any character **except** those inside the brackets.
One or More	**+**	Quantifier: Matches the preceding element **1 or more** times.
Zero or More	*****	Quantifier: Matches the preceding element **0 or more** times.
Optional	**?**	Quantifier: Matches the preceding element **0 or 1** time (makes it optional).
Specific Count	**{n}**	Matches exactly **n** repetitions.
Range Count	**{n,m}**	Matches between **n** and **m** repetitions.

Scent Discrimination & Stamina

(?i) This ensures the Bloodhound finds both "N" and "n". Your original code may miss the capital "N" in "Nothing," making your stats inaccurate.

Shorthand	Lowercase: The Target	Uppercase: "Not-Target"
\w vs \W	**Word Characters**: Letters, numbers, and underscores.	**Non-Word**: Punctuation, symbols, and spaces.
\d vs \D	**Digits**: 0, 1, 2, 3...	**Non-Digits**: Letters, spaces, and punctuation.
\s vs \S	**Whitespace**: Spaces, tabs, and newlines.	**Non-Whitespace**: Anything you can actually see/print.
\b vs \B	**Boundary:** r"\bcat\b" finds "the **cat**" but ignores "category."	**Not a Boundary:** r"cat\B" finds the "cat" inside "**cat**egory" but ignores the word "cat."

```
import re

poem = """ nothing can ever happen twice.
in consequence, the sorry fact is
that we arrive here improvised
and leave without the chance to practice. """

# Comparing the ratio of vowels to consonants
stat = len(re.findall(r"[aeiou]", poem))/len(re.findall(r"[b, c, d, f, g, h, j, k, l, m, n, p, q, r, s, t, v, w, x, y, z]", poem))
print(stat)
#Clean coding
stat = len(re.findall(r"[aeiou]", poem))/len(re.findall(r"(?i)[^aeiou\s\d\W]", poem))
print(stat)
```

Scent Discrimination & Stamina

```
import re
# Scenario: Finding words with repeated 'o' in a reaction transcript
transcript = "The subject said: 'Nooo! Noooo!'"

# The dog looks for 'No' followed by one or more 'o's
found = re.findall(r"No+", transcript)
print(found) # Output: ['Nooo', 'Noooo']
```

r"No+"	'N' + one or more **'o's**.	Nooo
r"N.+"	'N' + one or more **wildcards** (any char).	Nooo! N123! Nothing!

```
# Scenario: Finding 'Trial' followed by any number of digits (even 0)
log = "Trial, Trial1, Trial200"

star_results = re.findall(r"Trial\d*", log)
print(star_results ) # Output: ['Trial', 'Trial1', 'Trial200’]
plus_results = re.findall(r"Trial\d+", log)
print(plus_ results) # Output: ['Trial1', 'Trial200']
```

*	Quantifier: Matches the preceding element **0 or more** times.	**Zero or More**
+	Quantifier: Matches the preceding element **1 or more** times.	**One or More**

Scent Discrimination & Stamina

```
import re
# Scenario: Matching both singular and plural linguistic terms
comment = "The study of phoneme and phonemes."

# The 's' at the end is optional
terms = re.findall(r"phonemes?", comment)
print(terms) # Output: ['phoneme', 'phonemes']
```

```
import re

text = "We have one process and two processes."

matches = re.findall(r"process(?:es)?", text)
print(matches)
```

```
# Scenario: Finding specific IDs that are between 2 and 3 digits long
data = "Subjects: 1, 45, 600, 7000"

# The dog only barks for numbers with 2 to 3 digits
valid_ids = re.findall(r"\d{2,3}", data)
print(valid_ids) # Output: ['45', '600', '700']
```

{n}	**Exactly** n times.
{n,}	**At least** n times.
{n,m}	**Between** n and m times.

Greedy vs. Non-Greedy Logic

Logic	Syntax	Technical Description
Greedy Match	*** or +**	The default logic. It matches the **longest possible** string that fits the pattern.
Non-Greedy (Lazy)	***? or +?**	Adding a ? after a quantifier makes it "lazy." It matches the **shortest possible** string.
The "Capture All"	**.***	Matches any character, any number of times (Greedy).
The "Capture Minimal"	**.*?**	Matches any character, any number of times, but stops at the **first** opportunity.

```
import re
text = "[Subject_01] was fast, but [Subject_02] was faster."

# Greedy: Matches from the first '[' to the LAST ']'
greedy_result = re.findall(r"\[.*\]", text)

print(f"Greedy found: {greedy_result}")
# Output: ['[Subject_01] was fast, but [Subject_02]']
```

```
# Non-Greedy: The '?' tells the dog to stop at the FIRST ']'
lazy_result = re.findall(r"\[.*?\]", text)

print(f"Lazy found: {lazy_result}")
# Output: ['[Subject_01]', '[Subject_02]']
```

The Pattern Refiner

Regular Expression

a specialized *mini-language* used to describe search patterns in text

it allows you to perform complex text operations with a single line of code.
It is primarily used for:

Validation:

Ensuring a string follows a specific format

(e.g., "Is this a valid email address or phone number?")

Extraction:

Pulling specific pieces of information out of a large document

(e.g., "Find all the dates mentioned in these 1,000 transcripts").

Transformation:

Mass-cleaning data

(e.g., "Replace all inconsistent date formats with a standardized YYYY-MM-DD format").

The Pattern Refiner

re.sub()	re.sub(pattern, replacement, text)	**The Cleanup:** Searches for a pattern and **replaces** it with new text. Returns the modified string.

```
import re
tweet = "Linguistics is fun! #science #study #AI"

# The dog finds every '#' followed by word characters
# and replaces them with an empty string ""
clean_tweet = re.sub(r"#\w+", "", tweet)

print(f"Cleaned tweet: {clean_tweet}")
# Output: Linguistics is fun!
```

```
import re
comment = "Contact me at researcher_01@email.com."

# finds the email pattern and replaces it with [HIDDEN]
anonymized = re.sub(r"\w+@\w+\.\w+", "[HIDDEN]", comment)


print(f"Privacy version: {anonymized}")
# Output: Contact me at [HIDDEN].
```

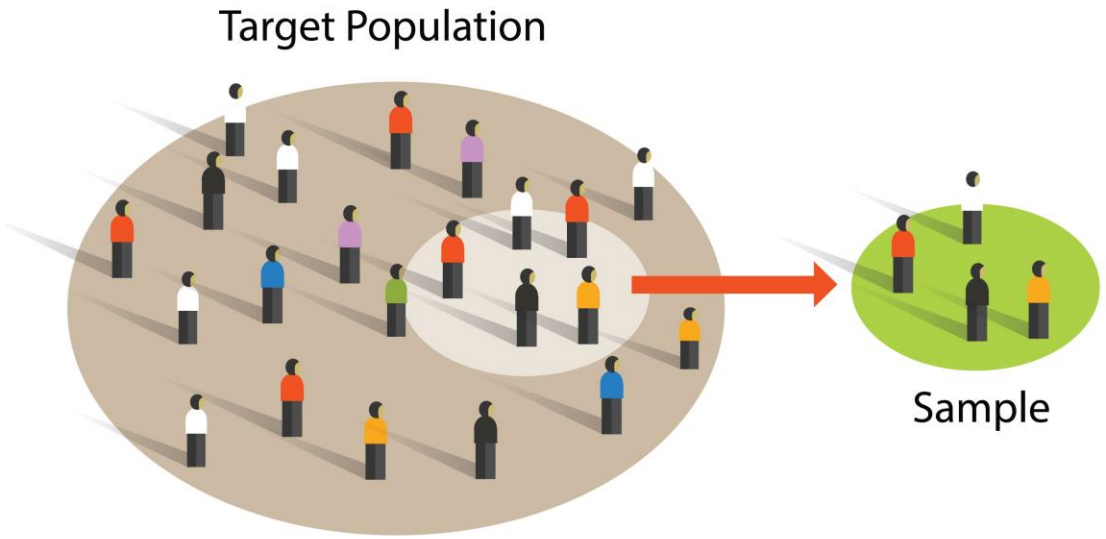
Capturing and Extracting

Capturing Group	**( )**	Parentheses isolate a part of the pattern to "capture" it in the memory.
Specific Group	**match.group(n)**	Returns the n-th captured group (1, 2, 3...).
All Groups	**match.groups()**	Returns a **tuple** containing all captured pieces.

```
import re

result = "Subject_05: Accuracy=92%"

# Group 1: (\d+) -> Captures the ID number & Group 2: (\d+) -> Captures the number for accuracy

pattern = r"(\d+): Accuracy=(\d+)%"
report = re.search(pattern, result)

if report:
    print(f"Extracted ID: {report.group(1)}")
    print(f"Extracted Score: {report.group(2)}")
```

The Variance of Truth

$$\sigma = \sqrt{\frac{\sum_{i=1}^{N}(x_i - \bar{x})^2}{N}}$$



$$s = \sqrt{\frac{\sum_{i=1}^{N}(x_i - \bar{x})^2}{N-1}}$$

Library	Function	Default ddof	Statistical Context
NumPy	np.std()	**0**	**Population**: Divides by N.
Pandas	df.std()	**1**	**Sample**: Divides by N-1.

This is why choosing your library matters. **NumPy** is built for pure linear algebra (where the matrix is the whole world), while **Pandas** is built for data science and statistics (where the table is just a window into the world).